

CS603 - Software architecture and Design

WEEK 1 – Software Design Concepts

Topic 1 Design (What & Why)

1. Design What & Why?

Design means making a plan before building something. Like an architect draws a blueprint before constructing a building, a software engineer designs the system before coding it.

The famous quote says it all: You can use an eraser on drafting table or a sledgehammer on the construction site. Fix mistakes during design not after coding.

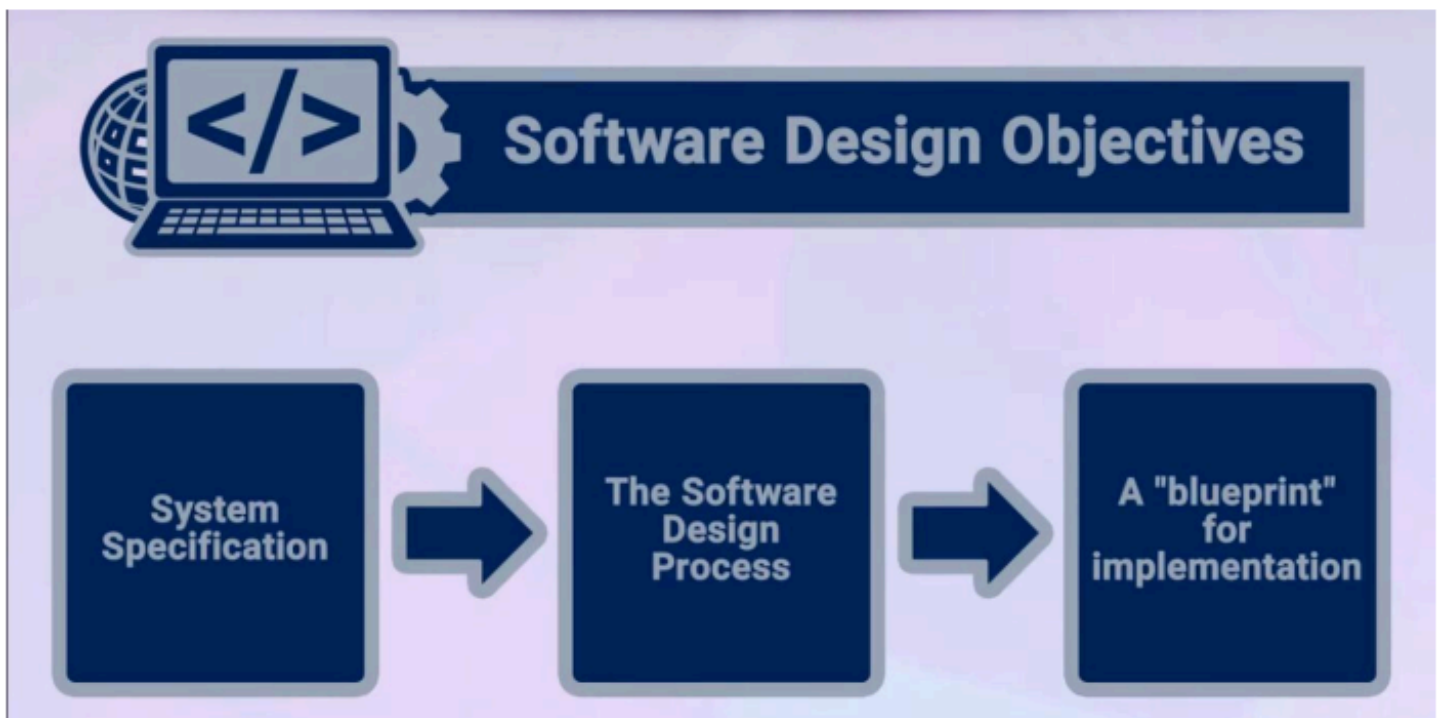
Example: Dress design, product design, building design, all need planing first. Software is no different.

2. What is Software Design?

Software Design is the process of defining the architecture, components, modules, interfaces, and data for a software system to satisfy specified requirements. It acts as a blueprint for software development and ensures that the system is structured, maintainable, and efficient.

It includes design principles, patterns, and methodologies to create high-quality software

Topic 2 Software Design (Objectives)



1. System Specification

System specification defines what the software should do.

It includes:

- Functional & Non-functional Requirement
- User Needs & Constraints
- Hardware & Software Requirements

2. The Software Design Process

The software design process is a structured approach that includes:

Requirement Analysis – Understanding what the system should do.

Architectural Design – Defining the system's high-level structure.

Detailed Design – Designing individual components/modules.

Validation & Verification – Checking the design to avoid errors.

3. A Blueprint For implementation
Software design acts as a blueprint (plan) that guides developers.
It includes:
Diagrams (UML, Flowcharts) to visualize system components.
Architecture patterns (MVC, Layered Architecture) to organize code.
Coding standards & guidelines to ensure quality
4. During the design phase , software engineers apply their knowledge of: □ The problem domain □ Implementation Technologies
5. Design Objectives:
Software design takes system specification (What the client wants) and turn into a Blue print for implementation (How to build it).
6. Goals of good software design
 - a. Meet the customers need
 - b. Be easy to implement
 - c. Be efficient
 - d. Be easily extendable for future needs.

Topic 3 &4 Software Design (Complexity &Size)

1. Remember that:
Poorly designed programs are difficult to understand & Modify. As you keep adding features to badly designed program, its get exponentially harder and more expensive.
A well designed program stays manageable even as it grows. Good Design = lower long-term cost.
2. Software design complexity refers to how difficult it is to understand, modify, test, or maintain a software system. It depends on:
Code Structure – Messy or well-organised?
Module Interactions – Are components highly dependent on each other?
Algorithms & Logic – Are they simple or too complex?
Scalability – Can it handle growth easily?
Example: A simple calculator app has low complexity, while an AI-based chatbot has high complexity due to advanced algorithms
3. **Software Design Size:**
Software size refers to the amount of code, number of modules, and overall system scope.
It is measured using:
Lines of Code (LOC) – Total number of written code lines.
Function Points (FP) – Measures functionality based on inputs, outputs, and interactions.
Cyclomatic Complexity – Measures the number of independent paths in the code.
4. **Impact of Size:**
Small-sized software is easier to develop and maintain.
Large-sized software requires careful planning, modularity, and documentation.
5. The bigger the program, the more severe the consequences of poor design. Size matters – a large badly-designed system becomes impossible to maintain

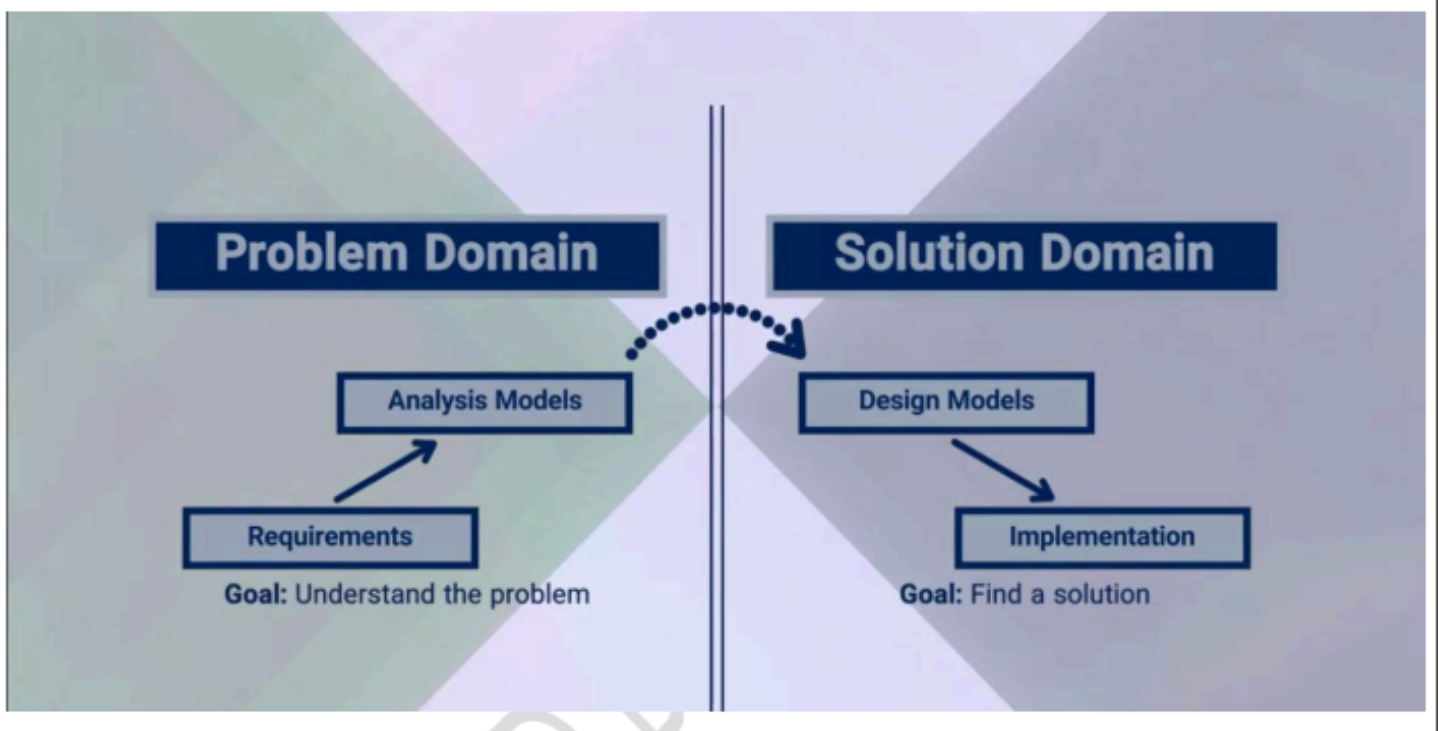
Topic 5 Software Design (types of Complexities)

1. **Essential Complexity:** Complexity that naturally comes from the problem itself. Cannot be avoided.

2. **Accidental/Incidental Complexity:** Complexity added due to poor design decisions. Can and should be avoided.
Total Complexity = Essential Complexity + Accidental Complexity
Good design reduces accidental complexity without removing essential complexity.
Example: A banking system is inherently complex (essential). Writing messy, tangled code makes it worse (accidental).
3. **Design: is an Antidote to complexity**
Design is a primary tool for managing essential & accidental complexities in Software.

Topic 6 Why is Software Design Hard?

1. Why is Software Design Hard?
Design is hard because you are creating an abstraction of a solution that does not exist yet. You are imagining and planning something before it is built.
You cannot fully see the problems until you try to solve them.



Topic 7 Software Design A science or an art?

1. Software design is both a science and an art. It follows scientific principles and methods, but also requires creativity, intuition, and experience. Two designers given the same problem will likely produce different designs – and both can be correct.
2. Software design is considered both a science and an art because it involves structured principles (science) and creative problem-solving (art).
 - Software design is a science because it follows engineering principles.
 - It is also an art because it requires creativity and problem-solving skills.
 - The best software designs balance both aspects for efficiency and user satisfaction.MCQ:
Science is a repeatable process. For Example: $2 + 2 = 4$

Topic 8 Software Design (A Wicked Problem)

1. **A Wicked Problem**

A problem that can only be clearly defined by completely solved it.

Software design is not just a step-by-step process; it requires continuous adaptation, problem-solving, and decision-making. Since there is no perfect solution, designers must focus on flexibility, scalability, and maintainability.

2. A wicked problem is one that can only be clearly defined by solving it. Design is a wicked problem – you only fully understand the problem after working through the solution. Even expert designers cannot always explain exactly how they arrived at their design. It comes through experience and iteration.

Topic 9 Design Process More Systematic & Predictable

1. **Making Design More Systematic**

To make the design process more predictable and repeatable, we use:

- Methods – structured processes to follow
- Techniques – specific tools and approaches
- Patterns – reusable solutions to common problems
- Principles and Heuristics – rules of thumb that guide good decisions

Topic 10 Dealing with Software Complexity Different rules & Principles

1. **Modularity, Hierarchy, Information Hiding, Abstraction**

Good design does not remove essential complexity but manages it so a programmer deals with only a small piece at a time.

Modularity: Break the solution into smaller, manageable pieces (divide and conquer).

Hierarchical Organization: Larger components are made of smaller ones (like folders containing files).

Information Hiding: Hide internal details behind simple interfaces. You use a TV remote without knowing its electronics.

Abstraction: Suppress unnecessary details. Focus only on what matters at that level.

Topic 11 Software Design (Characteristics)

1. **Non-deterministic:**

No two designers produce the same design. Unlike math, there is no single correct answer.

2. **Heuristic:** Design uses rules of thumb, not guaranteed formulas

3. **Emergent:** The final design evolves through experience, feedback, and iteration. It is not created in one step.

Deterministic Process: A Deterministic Process is one that produces the same output given the same inputs.

MCQ: Design is a non-deterministic Process. Because no two designers or design process are likely to produce the same output. **MCQ:** Design is Heuristic based Process. Because design is non-deterministic Process, design techniques tend to rely on Heuristics and rules-of-thumb rather than repeatable processes. Remember that: Design is a Emergent: The final decision evolves from experience and feedback. Design is an iterative and incremental process where a complex system arises out of relatively simple interactions

Topic 12 Software Design (Benefits of Good Design)

1. **Benefits of Good Design**

- Reduced complexity – easier to understand and modify
- Rapid development and easier maintenance
- Code reuse across projects
- Better software quality – easier to test
- Better security – complex code has more vulnerabilities

Benefits of Good Design:

- A good design reduces the Software Complexities making easier to understand and modify & implement.
- It make easy maintainance.
- Code Reuse
- It improves Software quality
- Good design exposes defects and makes it easier to test the Software

Topic 13 Software Design (A Generic Design Process)

1. A Generic Design Process The 6-step design process:
 - Step 1: Understand the problem (read requirements)
 - Step 2: Build a black-box model using use cases
 - Step 3: Look for existing patterns and solutions
 - Step 4: Discover missing requirements, build prototypes
 - Step 5: Document and review the design
 - Step 6: Iterate and refactor until all requirements are met

Processes are:

- **Software Requirements** (Understand the Problem)
- **System Specification** (Construct a “black-box” model of solution)
- **Architecture & Design Patterns** (Look for exiting solutions that cover some or all of the software design problems identified)
- **Discover missing Requirements**
- **Documents & review design**
- **Refactor** (iterate over solution)

Topic 14 Software Design (Inputs to the Design Process)

1. **User Requirements:**
What the client wants, including constraints on design and implementation.
2. **Domain Knowledge:**
Understanding the field (e.g., healthcare terms for a hospital application).
3. **Implementation Knowledge:**
Knowing the capabilities and limitations of the target technology or environment.

Topic 15 Desirable Internal Design (Characteristics)

1. **Minimal Complexity:** Keep it simple. Do not over-engineer
2. **Extensibility:** Design for today but keep the future in mind.
3. **Loose Coupling:** Minimize dependencies between modules.

4. **Reusability:** Build things that can be reused in other contexts.
 5. **Portability:** Works or can easily work in other environments
- Note:** Extensibility and minimal complexity can conflict. Engineering is about balancing such trade-offs.

We know that:

Poorly designed programs are difficult to understand & Modify.

&

Ease of maintenance is one of the prime objective.

Because

Your code will be read more often than it is written.

Desirable Internal Design:

- Minimal Complexity (Keep it Simple)
- Extensibility (Design for today but with an eye toward the future)
- Loose Coupling (Minimize dependencies b/w modules)
- Re-usability

WEEK 2 – Coupling, Cohesion and Design Strategies

Topic 16 Software Design (A Good Design)

1. Good design means Maintainable Design.

Specifically:

- Cost of making changes is minimal
- Easy to add or modify functionality
- Design is understandable by other developers
- Changes have local effect — do not break unrelated parts

This is achieved through High Cohesion and Low Coupling.

A Good design has:

- Efficient Code
- Compact Implementation
- Most Maintainable

In Maintainable Design:

- Cost of system changes is minimal
- Readily adaptable to modify existing functionality & enhance functionality
- Design should be understandable
- Changes should be local in effect
- High cohesion & low coupling

Topic 17 Software Design (Coupling).

Coupling

Coupling measures how dependent two modules are on each other. If changing one module forces changes in another, they are tightly coupled.

High coupling is bad: difficult to test, reuse, or modify modules independently.

Goal: Low coupling — make modules as independent as possible.

How to reduce coupling:

- Eliminate unnecessary relationships
- Reduce the number of necessary relationships
- Simplify the nature of necessary relationships

In other words:

Coupling:

The measure of dependencies b/w modules & components.

OR:

The degree of inter-dependence b/w modules & components.

A dependency exists b/w two modules/components if a change in one could require a change in the other.

1. *Basically: Coupling is an inter-component Relationship.*

2. *Measuring Coupling:*

The degree of coupling b/w two modules is determined by:

□ *The number of interfaces b/w modules (Quantity)*

□ *Complexity of each interface (determined by the type of communication quality)*

3. **Why high degree coupling is bad?:** (Imp)

□ *Highly coupled systems have strong interconnection.*

□ *High dependence of programs unit upon one another.*

□ *Difficult to test, reuse & understand separately.*

So: Loosely coupled modules are less dependent upon each other and hence are easier to change

4. **How to reduce Coupling?**

Coupling can be reduced by:

□ *Eliminating unnecessary relationships*

- Reducing the number of necessary relationships
- Easing the “tightness” of necessary relationships

Topic 18 Coupling – Example (Web App)

Example comparing two approaches for changing webpage style:

Option A (Bad): JavaScript directly modifies style attributes (color, font). Tight coupling between JS and CSS.

Option B (Good): JavaScript only changes the CSS class name. CSS handles all styling. Loose coupling.

Example: If you change the visual style in Option B, you only edit CSS. JS does not need to change at all.

Topic 19 Software Design (Cohesion)

1. Cohesion:

Cohesion measures how well the elements inside a single module belong together and work toward the same goal.

High Cohesion: All methods and data in a class serve one clear purpose. Good design.

Low Cohesion: Class does many unrelated things. Difficult to understand, test, and reuse.

Example: A class called User Manager that handles login, email sending, AND payment has low cohesion. Split into UserAuth, Email Service, and Payment Service.

Cohesion:

A measure of the strength of relationship b/w:

- The methods and data of a class and some underlying purpose of concept served by that class.
- The class’s methods and data themselves.

A module has high cohesion if all of its elements are working towards the same goal.

Low Cohesive System



And we can make it High Cohesion by:

High Cohesive System



2. **Highly Cohesive Modules are:**

- ☐ Easier to understand
- ☐ More Robust
- ☐ More Reliable
- ☐ More Reuseable

3. **Low Cohesive Modules are:**

- ☐ Difficult to understand
- ☐ Difficult to Maintain
- ☐ Difficult to Test
- ☐ Difficult to Reuse

Topic 20 Coupling & Cohesion (Relationship)

1. **Relationship Between Coupling and Cohesion**

They are inversely related: when coupling is high, cohesion tends to be low, and vice versa.

Best design = High Cohesion within a module + Low Coupling between modules.

Benefits: easier to read, modify, test, develop, and reuse.

As we know that:

Highly Cohesive Modules are:

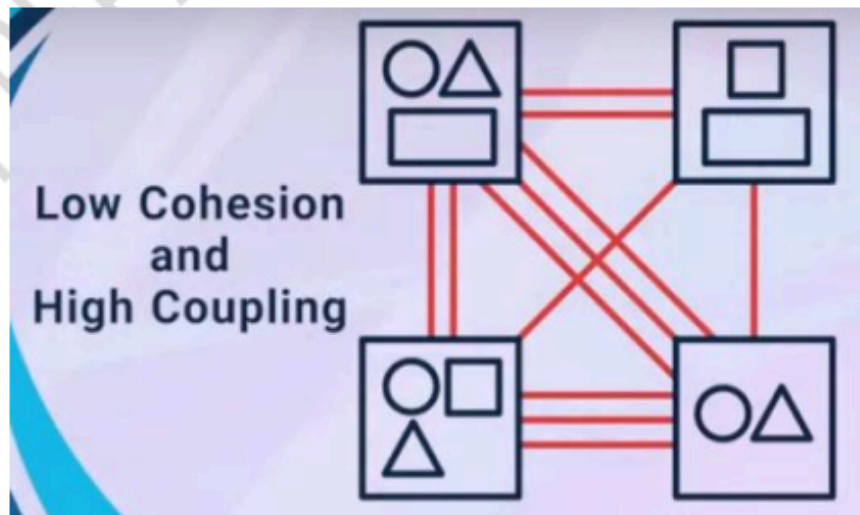
- ✓ Easier to understand
- ✓ More Robust
- ✓ More Reliable
- ✓ More Reuseable

&

Lowly Coupled Modules are:

- ✓ Easier to read & understand
- ✓ Easier to Modify
- ✓ More Reliable
- ✓ More Reuseable
- ✓ Easier to develop & test

You can see that both are SAME.



So:

Cohesion	Coupling
Intra-module Relationship	Inter-module Relationship

Topic 21 Software Design Strategies Structured & Object-Oriented Design

1. **Structured Design:**
System designed as a set of functions. System state is shared/centralized. Action-oriented approach.
2. **Object-Oriented Design (OOD):**
System is a collection of objects. Each object manages its own state and intelligence. Objects communicate through messages.
OOD is better for maintainability as it naturally supports low coupling and high cohesion.

Structured Design:

- ✓ System is designed from a functional view point.
- ✓ Conceived a set of functions that are used to solve a problem at hand.
- ✓ The system state is centralized and is shared among the functions operating on that state.

Object-Oriented Design:

- Based on the idea of information hiding.
- System as a viewed as a collection of objects.
- The system intelligence and state is decentralized and each object manages its own state information.

Attributes (State)

Operation (Intelligence)

- Objects communicate by exchanging information messages.

Conclusion:

Object-Oriented Design is to be considered best & more maintainable as compare to the Structured Design.

Topic 22 Object-Orientation Basic Concepts (Object)

1. The Object:

An Object is a real or abstract entity with a well-defined role in the problem domain.

Every object has three things:

State: Current values of its properties (e.g., a bank account's current balance).

Behavior: How it acts and reacts (e.g., deposit, withdraw methods).

Identity: What makes it unique from all other objects.

Topic 23 Object-Orientation Basic Concepts (Class)

1. The Class:

A Class is a template or blueprint for creating objects. All objects of a class share the same structure and behavior.

Interface (public view): What the class can do for you — the operations it offers to outside callers.

Implementation (private view): How it actually does it — the internal data structure and algorithm.

Example: 'Car' is a class. Your specific car with license plate ABC-123 is an object (instance) of that class.

WEEK 3 – OOP Relationships and UML

Topic 24 Relationship b/w Classes & objects (Association)

1. Association:

Association is a general relationship between two classes, meaning objects of one class are connected to objects of another. It does NOT mean all objects are always linked — just that a persistent connection can

exist.

Has attributes: name, direction (one-way or two-way), roles, and multiplicity (how many of each side).

Example: Student is associated with Course. One student can enroll in many courses.

Objects & Classes are two building blocks of the system.

Relationship b/w Classes:

- Behavior of the system is defined by the manner in which the objects interact with one another.
- In order to Construct a software system , we need mechanism that create connections b/w these building blocks.

Connections can be done by two ways:

Association

Inheritance

Association:

- ✧ It indicates that the objects of the two classes are related in some non-hierarchically way.
- ✧ It implies that an object of one class is making use of an object of another (or some) class.
- ✧ It indicated simply by a solid line connecting the two line icons.
- ✧ It does not imply that there is always a link b/w all objects of one class and all objects of the other.
- ✧ It does imply that there is a persistent , identifiable connection b/w two classes.

Remember that:

There is a Conceptual Link b.w Classes.

&

There is a Physical Link b.w Objects.

Attributes of Association:

- ✧ Descriptive Name
- ✧ Arity of the connection
- ✧ Direction
- ✧ Roles

Topic 25 Relationship b/w Classes & Objects Aggregation & Composition

1. Aggregation:

A whole-part relationship shown with a hollow diamond in UML. The part can exist independently of the whole. Most experts consider it a modeling placebo.

2. Composition

: A strong whole-part relationship shown with a filled diamond in UML. The part CANNOT exist without the whole. If the whole is destroyed, so are the parts.

Example: Composition: A House has Rooms — Rooms cannot exist without the House. Aggregation: A University has Students — Students exist independently.

Aggregation:

- ✓ It is a form of association that specifies a whole-part relationship b/w an aggregate (a whole) and a constituent part.

It is indicated by a "Hollow Diamond (→)"

There is a Sub-Class of aggregation which is called Composite Aggregation, also known as Composition.

Composition:

- ✓ It implies that each instance of the part belongs to only one instance of the whole, and the part cannot exist except as part of the whole.
- ✓ It is indicated by a "Filled-in-Diamond"

MCQs:

Aggregation: Loose relationship

Composition: Strong Relationship